D2.2d Workshop 2 — Speaker Script

HLD & LLD — D2.2d-HLD-LLD-001

Government of Sint Maarten · Orangebd · June 2026

Total duration: 45 minutes **Format:** Workshop with live Q&A

Audience: Stakeholders, registry owners, technical team, DLT

Timing overview

Title + Agenda + Expectations	—	5 min
User Journey + Design Principles	—	5 min
The Problem + Platform Overview	—	5 min
Components 01–05 (APIM, Adapters, CMF, Data, Events)	—	13 min
Components 06–09 (Identity, CI/CD, Security, Monitoring)	—	8 min
Component 10 — Payment Gateway (CX Pay & Sentoo)	—	2 min
Complete HLD	—	4 min
Q&A + Close	—	5 min

Notes in green boxes are reminders for the presenter. Text in italics is suggested spoken word — adapt freely to your natural voice.

Title Slide

Timing: 1 minute

Good morning everyone, and welcome to Workshop 2 of the D2.2d Digital Government Transformation Programme for Sint Maarten. I am glad you are all here.

Today we are going to walk through the High Level Design and Low Level Design of the Interoperability Platform — the technical backbone that will allow Sint Maarten's government registries to talk to each other, and allow citizens to interact with all of them through a single portal.

This is a working session. I want you to ask questions as we go, challenge what you see, and confirm or correct the details that only you can confirm. Your input today directly shapes what gets built.

Let us start with the agenda so you know what to expect.

Agenda Slide

Timing: 1 minute

We have organised today into three parts.

First, the context — we will begin with a live user journey showing exactly what Marcus, a sole proprietor, experiences when he registers his business through the new platform. Then we will look at the five design principles that guided every decision you will see today.

Second, the platform components — we will go through each of the ten technical components one by one. For each one I will show you a simplified diagram, explain what it does in plain terms, and give you a moment to discuss before we move on.

Third, we will bring it all together with the Complete High Level Design — a single view of the whole platform with everything connected.

We have about 45 minutes. I will keep pace but please stop me if something is unclear.

Expectations Slide (Mentimeter)

Timing: 3 minutes

Before we dive in, I want to take two minutes to hear from you.

On screen you can see a QR code. Please scan it with your phone — it will take you to a quick Mentimeter poll. The code is also on screen: menti.com, code 5346 4645.

You will see ten words. Pick any three that best describe what you are hoping to get out of today. There are no right or wrong answers — this just helps me make sure I address what matters most to this room. It is completely anonymous.

[Pause 60–90 seconds for responses]

Let us look at the results together.

[Read out top 3 results]

Good — that tells me [adapt based on results]. I will make sure we give those areas proper attention today.

Note: If Clarity and Confidence come up most, emphasise the plain-English explanations throughout. If Risks or Feasibility dominate, spend a little extra time on the resilience and open items sections.

User Journey — Marcus Registers His Business

Timing: 2 minutes

Let me introduce you to Marcus. Marcus is 34, born in the Antilles, residing in Sint Maarten. He wants to register his sole proprietorship, confirm his Business CRIB ID, and apply for an Economic Licence — all before Friday.

Today, that means visiting three separate government offices over three days. He already has all the required data — it exists electronically across multiple registries. The problem is that none of those registries can see each other.

On the screen you can see Marcus's journey through the new platform. Seven steps. One portal. One login. Let me walk you through them quickly.

Step one — he logs in with his national eID. One click. No new account, no new password.

Step two — his details appear automatically. Name, date of birth, address — already filled in. The platform read them from the Civil Registry.

Step three — his business is registered in COCI Portal. Seconds.

Step four — his Tax ID is confirmed automatically. No WhatsApp message, no email to the Tax Administration.

Step five — his Economic Licence application is submitted to TEATT.

Step six — his address is validated against the official Address Registry.

And step seven — he is done. Five registries. One session. Zero office visits.

That is what this platform enables. Everything else we discuss today is the architecture that makes this possible.

Design Principles

Timing: 2 minutes

Before we look at the components, I want to share five principles that guided every decision in this design. These are not aspirational statements — each one has a direct expression in the architecture.

The first is Open, Standards-Based Integration. Every registry connection uses a published, open protocol. No proprietary lock-in. If a vendor changes, only the relevant adapter or configuration changes.

The second is Authoritative Source, Loosely Coupled. Each registry remains the owner of its own data. We do not copy or migrate anything permanently. And each connector is independent — if one registry goes down, the others continue working.

The third is Security, Identity, and Privacy by Design. Security is not added on top. Every request passes through eight security gates. Citizens can only access their own data. These rules are enforced at the architecture level.

The fourth is Resilience and Once-Only. Citizens provide their information once. The platform fetches it. A built-in mechanism called the Outbox guarantees no data is ever lost, even if the system crashes mid-transfer.

And the fifth is Observable and Reproducible Infrastructure. Every component logs structured data with a unique tracking ID. All infrastructure is code — a new engineer can reproduce the entire platform from a single pipeline run.

These five principles explain why things are built the way they are. Keep them in mind as we go through each component.

The Problem — What Challenges Do Citizens Face?

Timing: 1.5 minutes

Let us look at the problem we are solving.

On the left you can see what Marcus wants — register his business, confirm his CRIB ID, apply for an Economic Licence. Simple.

On the right you can see what it takes today. Three separate steps. Visit COCI in person. Email or WhatsApp the Tax Administration and wait for a manual reply. Then submit to TEATT separately.

Three days. Three agencies. Zero shared data.

The data already exists — it is in the registries. The problem is that it lives in silos, and there is no bridge between them.

That is the gap this platform is designed to close.

Platform Architecture — How the Platform Closes the Gaps

Timing: 2 minutes

The platform closes three structural gaps.

On the left you can see the seven-layer architecture stack. Each layer has a specific job. Click any layer and you will see the detail — but for now let me summarise what this means in practice.

Gap one is now closed. Every registry shares a common set of identifiers — a citizen ID, a business ID, a parcel ID, and an address key. One request to the platform returns a joined record across all registries. No separate logins, no re-entering the same information.

Gap two is now closed. The citizen logs in once with their national eID. That single login gives them secure access to their own records across all six registries. No manual emails, no WhatsApp.

Gap three is now closed. Each registry uses a different technical language. The platform translates all of them behind the scenes. The portal sends one request and gets back one unified response.

The seven layers we are about to walk through are the engineering that delivers these three outcomes.

Component 01 — APIM Gateway

Timing: 2.5 minutes

The first component is the APIM Gateway. Think of this as the front door of the platform. Every single request — from any citizen, from any portal — passes through this gateway before it can reach a registry.

The gateway runs eight sequential stages. Any stage can stop the request entirely. Only a request that passes all eight reaches a registry.

Let me walk through them in plain English.

Stage one: Are you logged in? A valid login token is required. If not, the request is blocked.

Stage two: Are you allowed to do this specific operation? The gateway checks the permission attached to your login.

Stage three: Are you going too fast? A rate limit prevents any single user or system from overloading the platform.

Stage four: Are you coming from a trusted location? An IP filter blocks requests from unknown sources.

Stage five: Is your request properly formed? Malformed requests are rejected before they can do any damage.

Stage six: A unique tracking ID is stamped on every request. This is what allows us to trace a single citizen's journey through every log across every component.

Stage seven: The gateway authenticates itself to the registry it is about to call.

And stage eight: The request is forwarded.

Eight gates. Every request. Every time.

I encourage you to click any stage on the slide to see the technical detail behind it.

Component 02 — Registry Adapters

Timing: 2.5 minutes

The second component is the Registry Adapter Layer.

Each of the six government registries speaks a different technical language. The Civil Registry uses an old SOAP protocol. The Business and Taxpayer Registries use a modern Microsoft CRM standard. The Address and Land Registries use a GIS mapping system.

Rather than asking the portal to speak all six languages, we built a separate connector for each registry. Each connector translates its registry's native language into a single Common Format before the result ever reaches the portal.

All six connectors share a common engine. That engine handles five things for every registry: read the response, translate it into the common format, retry automatically if the registry is slow, handle errors consistently, and log everything — without logging any citizen data.

The key insight: the registries do not talk to each other. They each talk to this engine, which speaks all their languages.

One important note on the Land Registry — we are waiting for their technical documentation before we can complete that connector. I would like to ask today: can we confirm those technical details in this session?

Note: Pause here and look at the Land Registry stakeholder in the room. This is a workshop action item.

Component 03 — Common Message Format (CMF)

Timing: 2 minutes

The Common Message Format is the contract between the adapters and everything above them.

Here is the problem it solves. The Civil Registry stores a surname as achternaam. The Business Registry might store it as ownerSurname. The CMF defines one agreed name for each piece of data, and each adapter translates the registry's name into the CMF name before anything reaches the portal.

The portal only ever reads the CMF. It never sees the registry's original field names.

Four identifiers run through every CMF record and link all six registries to the same citizen: a Citizen ID, a Business ID known as the CRIB number, a Land Parcel ID, and an Address Key. Because every registry recognises these four identifiers, a single request can return a citizen's civil, tax, business, and address data in one go — without asking the registries to talk to each other.

Once fetched, CMF records are stored temporarily in the platform's own database — Cosmos DB. They expire automatically based on how often that type of data

changes. Business status expires after five minutes because it can change quickly. Addresses expire after 24 hours because they change rarely.

Below the diagram you will see the proposed field name mapping. These are what the platform proposes to call each field and what we believe the registry currently calls it. Every registry owner in this room needs to confirm whether these names are correct. Without that confirmation, the connectors cannot be built.

Can we go through these today?

Note: This is the most important workshop action. Each registry owner should review their tab in the CMF table and confirm or correct the field names. Allow a few minutes for this if the room is engaged.

Component 04 — Data Layer

Timing: 2.5 minutes

Before I describe the data layer, I want to be clear about one thing: the registries keep their own databases. Nothing is migrated. Nothing is copied permanently. The Civil Registry, the Business Registry, and all others continue running on their own systems exactly as before. The platform only stores what it fetches and translates. The registries remain the single source of truth.

The platform uses three separate stores, each with a specific purpose.

Azure SQL holds permanent records. The Outbox staging area, which ensures no data is lost. The Audit Log, which records every data access permanently for seven years, with citizen identities stored as scrambled codes. And the Registry Code Dictionary, which translates system codes like ONGEHUWD into readable labels.

Cosmos DB holds the temporary CMF copies. When Marcus requests his registration, the platform fetches his record, translates it, and stores it here temporarily. It expires and is deleted automatically.

Redis holds only technical credentials. Login tokens for the mapping system, fast-access copies of the code dictionary, and request counters that help enforce rate limits. There is one hard rule about Redis: no citizen personal data, ever.

The next slide shows a concrete example — what actually gets written to each store when Marcus applies for his National ID.

Component 04 — Data Layer Example (National ID)

Timing: 1.5 minutes

This slide makes the data layer concrete.

Marcus applies for his National ID. Here is exactly what the platform writes.

In Azure SQL: an Outbox entry is created the moment the adapter saves his data — same operation, same instant. An Audit Log entry is written: his identity as

a scrambled code, which registry was queried, the timestamp. No readable name, ever.

In Cosmos DB: his citizen profile — name, date of birth, civil status, fetched from the Civil Registry. His validated address. His tax clearance status. All three expire automatically.

In Redis: the GIS mapping token the platform uses to connect to the Address Registry. A request counter noting how many calls this session has made.

What is not here: Marcus's actual name, his address, or any personal detail in readable form in Redis or the Audit Log. Those exist only in the Civil Registry's own database and temporarily in Cosmos DB.

I also want to address a question that often comes up: why do different records have different expiry times rather than a single uniform setting? Because different data changes at different rates. Business status can change within minutes — a company can be suspended or reinstated quickly. A home address rarely changes. The expiry time is tuned to the nature of the data, not chosen arbitrarily. The proposed expiry times are on the screen — I would like each registry owner to confirm whether these are appropriate for their data.

Component 05 — Events (How the Platform Keeps Data Up to Date)

Timing: 2 minutes

When a registry changes a record, the platform needs to know — and update its copy — without losing any data along the way.

Here is how it works, step by step.

Step one: a registry updates a record.

Step two: the adapter fetches the updated record and translates it into the Common Format.

Step three: the adapter writes the translated record and a dispatch note at the exact same instant, in a single operation. This is what we call the Outbox. If the system crashes at any point after this, the dispatch note is still there when it recovers. No data is lost. Only time is lost.

Step four: a relay process runs every few seconds, finds any pending dispatch notes, and forwards them.

Step five: the message dispatcher sends the update to three listeners simultaneously.

The first listener writes a permanent audit log entry. The second refreshes the platform's CMF copy — the citizen portal sees the change within 15 minutes. The third listener will send a push notification to the citizen's portal session — but this is Phase 2, not active at launch.

There is also a safety net: if any message fails to deliver and sits undelivered for more than three days, the operations team is automatically alerted. This is treated

as an incident.

No update is ever lost. No data is shared between registries. Every access is permanently logged.

Component 06 — Identity

Timing: 2 minutes

I want to be precise about the identity layer, because there is a common misunderstanding worth addressing directly.

Azure Entra External ID is part of this platform — we are building it. The national eID is not part of this project. It is owned and operated separately by the government. Our responsibility is the integration point.

Here is how the integration works.

Step one: a citizen taps Login on the portal. They are redirected to the national eID login page. The platform never sees their password or credentials.

Step two: the eID authenticates the citizen and sends a confirmation back using a standard protocol called OIDC. Azure Entra External ID — the component we are building — receives this confirmation and converts it into a short-lived session token. The eID's job ends here.

Step three: that session token is issued to the citizen's browser. It identifies them for the rest of their session.

Step four: every request to the APIM gateway is validated against this token at stage one of the eight-stage pipeline we discussed earlier.

Two design choices worth highlighting.

The platform is not tied to any specific eID provider. If Sint Maarten switches providers in the future, only the Entra configuration changes. No adapter, no portal, no connector is touched.

Citizens can only access their own data. The session token identifies who is making the request. Certain registries require this token before returning any data. A government system cannot use this route to access a citizen's records.

One open item: a legal ordinance recognising eID as a valid authentication method for government services must be in place before the platform can go live. I would like to confirm the current status of that ordinance today.

Component 07 — CI/CD

Timing: 2 minutes

Every change — whether to the application code or the cloud infrastructure — goes through the same principle: automated checks first, then a human gate before anything reaches production.

There are two parallel pipelines.

The application pipeline has five stages. Build and test, which runs on every code change and includes a security scan for vulnerabilities. Integration testing, which answers the question: do the components work together? The APIM gateway, adapters, and CMF format are tested as a connected system. Deployment to the development environment. Staging, which answers the question: did the new change break anything that was already working? And finally, production.

The infrastructure pipeline follows the same pattern for cloud resource changes. Plan, apply to development, apply to staging, apply to production.

Two rules apply to both pipelines.

First, we operate a pipeline-first approach with human intervention permitted when needed. All routine changes go through the pipeline. Manual intervention by authorised engineers is allowed for emergencies, but must be documented and followed up with a pipeline-based fix.

Second, every change can be rolled back. Production deployments are tagged with a rollback reference. If something goes wrong, the previous state can be restored.

Component 08 — Security

Timing: 2 minutes

Security in this platform is layered. The diagram shows five concentric circles — think of them as five lines of defence, each one catching what the previous one missed.

The outermost layer is the Perimeter: Front Door WAF and DDoS Protection. Malicious traffic is blocked before it even reaches the platform.

The Network layer ensures all internal services communicate over private connections — never the public internet.

The Application layer runs every request through the eight-stage APIM gateway we discussed, and scans every code change for security vulnerabilities automatically.

The Identity layer enforces that every call carries a valid login token and that citizens can only access their own data.

And at the core, the Data layer stores all secrets and credentials in Key Vault. Services access them only through their assigned identity. No plaintext credentials anywhere.

I also want to explain why the security monitoring system is deliberately phased. At launch, the system monitors and observes. It does not automatically block or isolate services. The reason is simple: automated response is only safe after alert thresholds have been tuned and validated. We do not yet know what normal traffic looks like for this platform. Phase one is the tuning period. Phase two activates automated response once quality is confirmed. Phase three hands monitoring over to GoSXM.

Component 09 — Monitoring & Observability

Timing: 1.5 minutes

Every component in the platform emits structured logs. Every log line carries the same six fields: a tracking ID that links every entry for one citizen's request across all components, which registry was involved, what action was performed, how long the registry call took, the citizen's identity as a scrambled code, and the environment.

That tracking ID is what allows us to trace a single request from the moment a citizen taps a button to the moment the response arrives — across every component and every registry.

On the right you can see the alert rules. These are the conditions that automatically notify the operations team.

I want to highlight that all thresholds are proposed. They will be confirmed and tuned during Phase 1 operations once we have real traffic to observe. The teams responsible for responding to each alert will also be confirmed by stakeholders — we have left those assignments open for now.

Component 10 — Payment Gateway

Timing: 2 minutes

The final component is the payment gateway integration. When a citizen completes a service that requires a fee — submitting an Economic Licence application, for example — the platform routes the payment through one of two integrated providers: CX Pay or Sentoo.

I want to be clear about something important before we look at the diagram. The platform does not process payments. It does not store card numbers. It does not handle funds. All of that is the payment provider's responsibility. What the platform does is initiate the payment request, hand the citizen over to the provider's secure checkout, receive a signed confirmation when payment is complete, and then continue the service.

The diagram shows the four steps. Step one: the citizen completes their service and a fee is required. Step two: the platform redirects the citizen to the provider's secure checkout. Step three: once payment is confirmed, the provider sends a signed callback to the platform. The platform verifies that signature, writes a transaction reference and payment status to the Audit Log, and then step four: the service continues automatically.

Let me briefly describe the two providers.

CX Pay is a global payment provider — cxpay.global. It supports card payments, international transactions, and multi-currency settlement. Suitable for government services where citizens may be paying from overseas.

Sentoo is a Caribbean payment network — sentoo.io — with explicit Sint Maarten support. It offers Pay by Bank through local bank accounts, credit card payments,

iDEAL, and Wero. It is used across Sint Maarten, Curaçao, Aruba, Bonaire, Saba, and St. Eustatius.

Both providers handle PCI-DSS compliance entirely on their side. The platform is completely out of PCI scope.

One open item: API credentials and webhook endpoint URLs for both providers need to be confirmed before Sprint 1 begins.

Note: Pause and ask: Is the preference to offer both providers at launch, or start with one and add the second later? This is a product decision that affects the portal team's Sprint 1 scope.

Complete HLD — High Level Design

Timing: 3 minutes

This slide brings everything together. This is the complete High Level Design for the D2.2d Interoperability Platform.

On the left you have the key numbers. Seven architecture layers. Six government registries. One Common Message Format. Four cross-registry keys.

In the centre you have the seven-layer stack. Each layer is clickable — if you want to understand what any layer does in detail, tap it and the panel at the top of the slide will show you the full description, the technology used, and any open items.

Below the stack you can see all six registries: Civil, Business, Taxpayer, Economic Licenses, Address, and Land. Click any registry box to see what data it holds, which protocol it uses, and what the adapter needs from the registry owner to be built.

On the right you have three summary panels: the citizen journey showing Marcus's six-step registration, the resilience guarantees, and the data security rules.

I want to give you a moment to look at this slide and ask questions. This is your architecture. Does it reflect the system you were expecting? Is there anything you see that needs to be corrected or confirmed?

[Pause for questions and discussion — allow at least 2 minutes]

Questions & Answers — Close

Timing: 5 minutes

Before we close, let me summarise the open items we need confirmed from today's session.

First, the CMF field names. Every registry owner needs to confirm whether the proposed field names in the mapping table match what your system actually exposes. Without this, the connectors cannot be built.

Second, the proposed data expiry times in Cosmos DB. Are five minutes for business data and fifteen minutes for civil and tax data appropriate for your registries?

Third, the Land Registry technical documentation — can we confirm the technical details needed to build that adapter?

Fourth, the status of the eID legal ordinance. The technical integration can be built now, but it cannot be switched on without the legal framework in place.

And fifth, the alert rule thresholds and team assignments for the monitoring system.

And sixth, API credentials for both CX Pay and Sentoo — needed before Sprint 1 to build and test the payment integration. The team also needs to decide: do we launch with both providers, or start with one?

These are the decisions that unblock the build phase. Everything else is designed and ready.

Thank you very much for your time today. I am now happy to take any questions.

Note: After Q&A, offer to walk through any specific slide in more detail. Keep the Mentimeter poll in mind — if *Confirmation* was a top response at the start, revisit whether those open items have been addressed.
